DEPI – Software Testing Track

Part 01: Java Fundamentals

Instructor: Mina Younan

Session 09 – Task A

Question: Create a Java Maven console application to implement the following scenario for an e-commerce inventory and shopping cart management system.

Scenario Overview

You are developing a backend console application for an online store. The store sells two main categories of products: **Clothing** and **Devices**. Each specific product type has unique properties, but all products share common behavior and attributes defined by an abstract **Item** base class. Customers can browse inventory, add items to a shopping cart, view items' details dynamically, and check out.

Technical & Class Structure Requirements

1. Core Object-Oriented Hierarchy

Implement the following class hierarchy using proper encapsulation (private fields, getters/setters, constructors):

- **Item (Abstract Base Class)**
  - **Attributes:** name (String), price (double), availableCopies (int)
  - **Methods:**
    - sell(): Decrements availableCopies by 1 if stock is available; throws an error or displays a message if out of stock.
    - returnItem(): Increments availableCopies by 1.
    - getCategory(): *Abstract method* returning a String representing the category name.
- **Clothing (Abstract Class — extends Item)**
  - **Attributes:** size (String), color (String)
- **Device (Abstract Class — extends Item)**
  - **Attributes:** brand (String), warrantyMonths (int)
- **Concrete Clothing Classes (extend Clothing):**
  - Shirt: Adds material (String). Implements getCategory() to return "Clothing - Shirt".
  - Socks: Adds pairsInPack (int). Implements getCategory() to return "Clothing - Socks".
  - Hat: Adds style (String). Implements getCategory() to return "Clothing - Hat".
- **Concrete Device Classes (extend Device):**
  - Printer: Adds isColor (boolean). Implements getCategory() to return "Device - Printer".
  - Laptop: Adds ramGB (int). Implements getCategory() to return "Device - Laptop".
  - Projector: Adds lumens (int). Implements getCategory() to return "Device - Projector".

2. Shopping Cart System

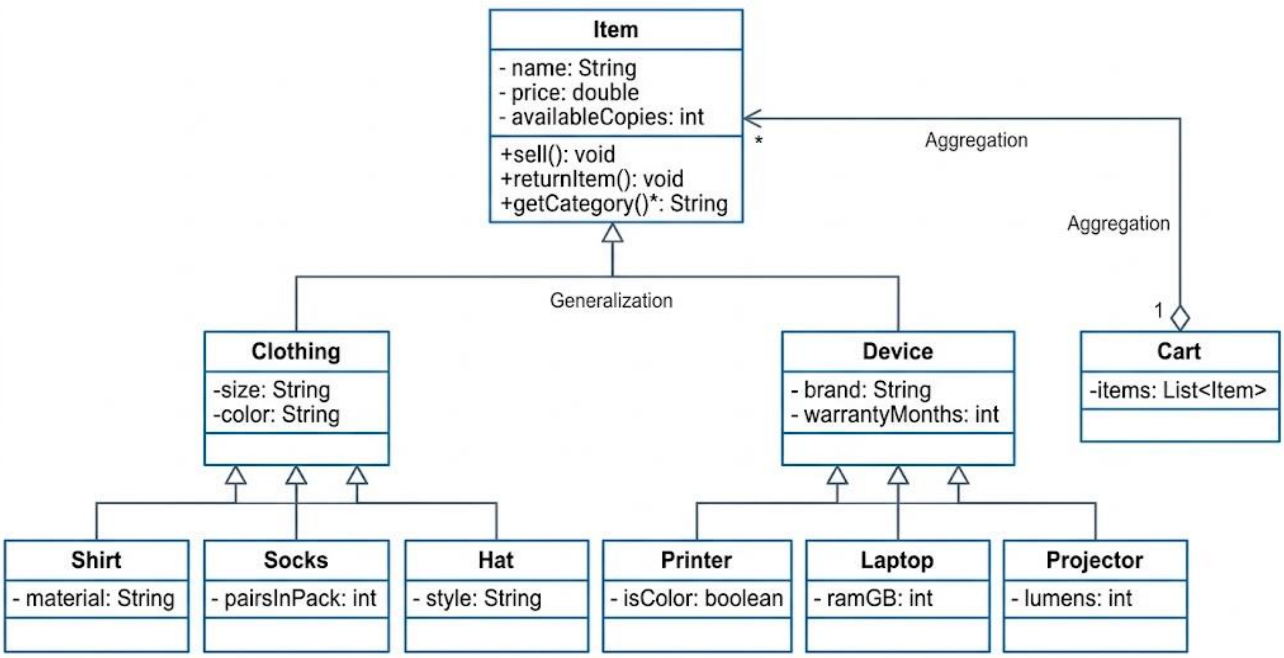
- **Cart Class**
  - **Attributes:** items (List<Item>)
  - **Methods:**
    - addItem(Item item): Adds an item to the cart.
    - removeItem(Item item): Removes an item from the cart.
    - calculateTotal(): Calculates and returns the total cost of all items in the cart.
    - checkout(): Invokes sell() on every item in the cart and clears the cart upon success.

Console Application Flow (Main.java)

Your application must present an interactive command-line interface (CLI) using java.util.Scanner with a menu loop offering the following choices:

1. **View Store Inventory:** Display a pre-populated list of items with their available stock, category (using polymorphic calls to getCategory()), and specific attributes.
2. **Add Item to Cart:** Allow the user to select an item from the store inventory by ID/index and add it to their Cart.
3. **View Cart:** Display all items currently in the cart, calling getCategory() and formatting each item's details, along with the total price.
4. **Checkout:** Process all items in the cart, reducing inventory counts (sell()), displaying an itemized receipt, and resetting the cart.
5. **Return Item:** Allow the user to return an item back to inventory (returnItem()), increasing its stock.
6. **Exit:** Terminate the program.

Solution Hint



Level	Class	Clear attributes
Abstract base	**Item**	name, price, availableCopies
Abstract middle	**Clothing**	size, color
	Shirt	material
Concrete clothing	Socks	pairsInPack
	Hat	style
Abstract middle	**Device**	brand, warrantyMonths
	Printer	isColor
Concrete devices	Laptop	ramGB
	Projector	lumens

Summary of Class Diagram Elements

1. **Abstract Classes (<<abstract>>)**
   - **Item**: Root class containing general properties (name, price, availableCopies) and methods (sell(), returnItem(), and abstract getCategory()).
   - **Clothing**: Extends Item; holds size and color.
   - **Device**: Extends Item; holds brand and warrantyMonths.
2. **Concrete Subclasses**
   - **Clothing Types**: Shirt (material), Socks (pairsInPack), Hat (style).
   - **Device Types**: Printer (isColor), Laptop (ramGB), Projector (lumens).
3. **Relationships & Notation**
   - **Generalization (Inheritance - ◁—)**: Solid line with an hollow arrow pointing from child to parent (Clothing → Item, Shirt → Clothing, etc.).
   - **Aggregation (o--)**: Hollow diamond on Cart side pointing to Item, with multiplicity 1 (Cart) to 0..* or * (Items).